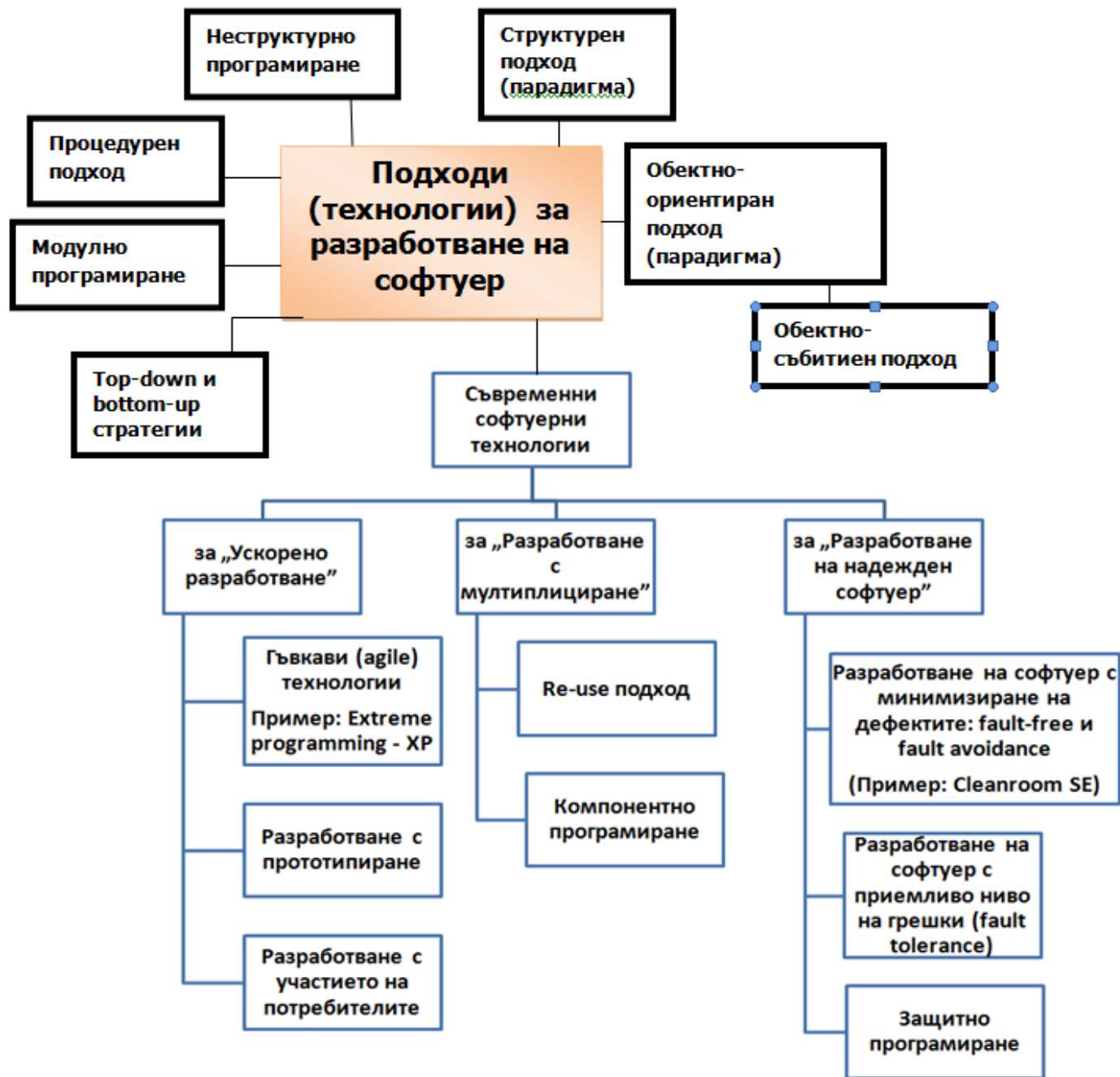


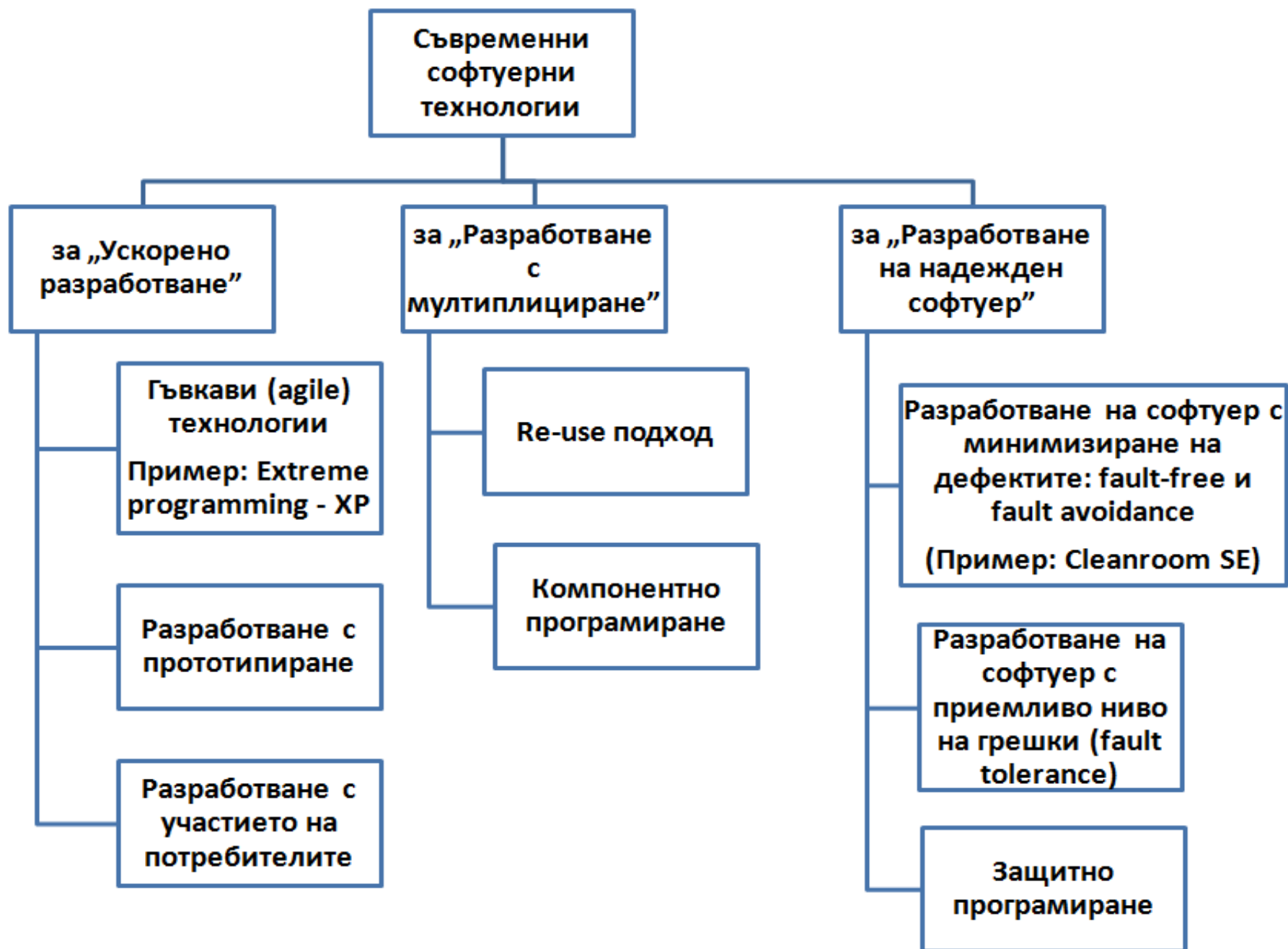
Тема 3. Съвременни технологии за разработване на софтуер

дисциплина: Технология на софтуерното производство
лектор: доц. В.Божикова

Съдържание

- **Технологии за „Ускорено разработване“**
 - Гъвкави (agile) технологии (Пример: *Extreme programming - XP*)
 - Разработване с прототипиране
 - Разработване с участието на потребителите
- **Технологии за „Разработване с мултиплициране“**
 - Подход с повторно използване - ППИ (Re-use подход)
 - Компонентно програмиране
- **Технологии за „Разработване на надежден софтуер“**
 - Разработване на софтуер с минимизиране на дефектите в него: подходи ***fault-free*** и ***fault avoidance*** (Пример: *Cleanroom SE*)
 - Разработване на софтуер с приемливо ниво на грешки (fault tolerance)
 - Защитно програмиране





1. Технологии за “Ускорено разработване”



1.1. Гъвкави (agile) технологии

Причини за поява:

- Промените в бизнес средата за разработване и в комерсиална реализация на софтуер, след 90-те години на миналия век;
- Непрекъснато променящи се пазарни условия;
- Променено отношение на софтуерните разработчици към стила на работа
- и др.

Основни принципи

5 са основните принципи на гъвкавите технологии:

- **Активно включване на потребителите** в процеса на разработване на Софтуера;
- Реализация на „**разрастващо се**“ (**инкрементално**) **разработване**, като изискванията за всяка следваща итерация се преформулират въз основа на оценяване на текущата;
- Регламентиране на **стил на работа, основан на компетентност, мотивираност и екипност**; стимулиращ творчеството психологически климат;
- **Приемане на неизбежността на промените**, съпътстващи софтуерните проекти и адаптиране на техниките на разработване към тях;
- Придържане към опростени конструкции и схеми на работа, с цел **минимизиране на ресурсите** за осъществяването им.

Множество гъвкави технологии

В съответствие с тези 5 принципа са създадени различни технологии за разработване на софтуер:

- Екстремно програмиране (Extreme programming - XP)
- Бързо (RAD - Rapid Application Development) разработване на софтуер
- Адаптивно разработване на софтуер
- SCRUM и др.

Като цяло:

- Гъвкавите технологии не предлагат универсално решение на проблемите в софтуерното производство.
- Те са подходящи, ако в организацията е налице нужният стил на работа, при това за разработка на не „критичен“ софтуер.

Пример: Extreme programming - XP

Определение - лека методология за малки и средни колективи, разработващи софтуер при **неясни** или **бързо променящи се изисквания**.

Extreme programming:

Определени принципи (12 на брой, изброени по-нататък) и практики се довеждат до възможния си предел (екстремум).

- **итерациите са непрекъснати,**
- **непрекъснато тестване,**
- **непрекъснатата преработка на архитектурата и кода,**
- **непрекъснато участие на клиента...**

Цел:

- **Скоростно получаване на конкретен резултат;**
- **Удовлетворяване основните изисквания на възложителя;**
- **Сравнително по-бавна доработка на системата в съответствие с допълнителните изисквания на потребителя.**

Характерни особености на ХР

- а) **Кратки итерации** (около седмица), с цел - отреагиране на значителни изменения в началните изисквания.

Забележка:

Това е технология, която има за прототип водопадния модифициран модел, но предполага итерации, всяка от които се състои от 4 фази: анализ на изискванията, проектиране, програмиране и тестване.

- б) Развитието на системата се разглежда преди всичко от гледна точка на потенциалните рискове за достигане на **крайния срок** и **вмъкване в заложения бюджет**.

- в) Предполага наличие на:

- **високопроизводителен колектив, с добри взаимоотношения** и
- **възможност на отделните участници да изпълняват разнообразни дейности.**

Основни принципи (12 на брой) на ХР

1. **Взаимодействие:** Всички членове на екипа трябва да умеят да синхронизират работата си, като се съобразяват със структурата на целия проект.
2. **Избор на приоритети:** Заедно с възложителя се решава, коя функция трябва да се реализира задължително и коя отсрочена.
3. **Малки реализации:** Разработчиците създават работоспособна версия много бързи и разширяват възможностите и много често.
4. **Система за именоване:** В процеса на разработване и общуване се използва единна терминология.
5. **Опростен продукт:** Създаваната програма трябва да бъде опростен продукт, отговарящ на изискванията на потребителя. В него не се залагат бъдещи възможности.

6. **Тестването, като методически подход:** В рамките на целия проект усилията са съсредоточени за проверка на създаваната система. Програмистите предварително подготвят тестове, на база изискванията на възложителя. След което пишат програмен код, който трябва да отговаря на съответния тест.
7. **Преоценка на потребностите:** Структурата на продукта постоянно се подобрява, като се избягват повторения и реализация на непотребни възможности.
8. **Работа по двойки:** Програмите се пишат по двойки – единия от програмистите пише кода, а другия обмисля архитектурата.
9. **Колективна собственост:** Целият код принадлежи на всички програмисти. Всеки може да внася съгласувани изменения във всяка част. Използва се единен стил.

- 10. Непрекъснатата итерация:** Събирането на продукта се реализира многократно в рамките на предварително планирани итерации. Така се ликвидират множество проблеми, свързани със интегрирането на модули, разработвани от различни програмисти.
- 11. 40-часова работна седмица :** Уморените програмисти правят много грешки.
- 12. Потребител на разположение:** Във всеки момент е налице специалист от страна на възложителя, който може да уточни дадено изискване, да смени приоритет или да отговори на конкретен въпрос.

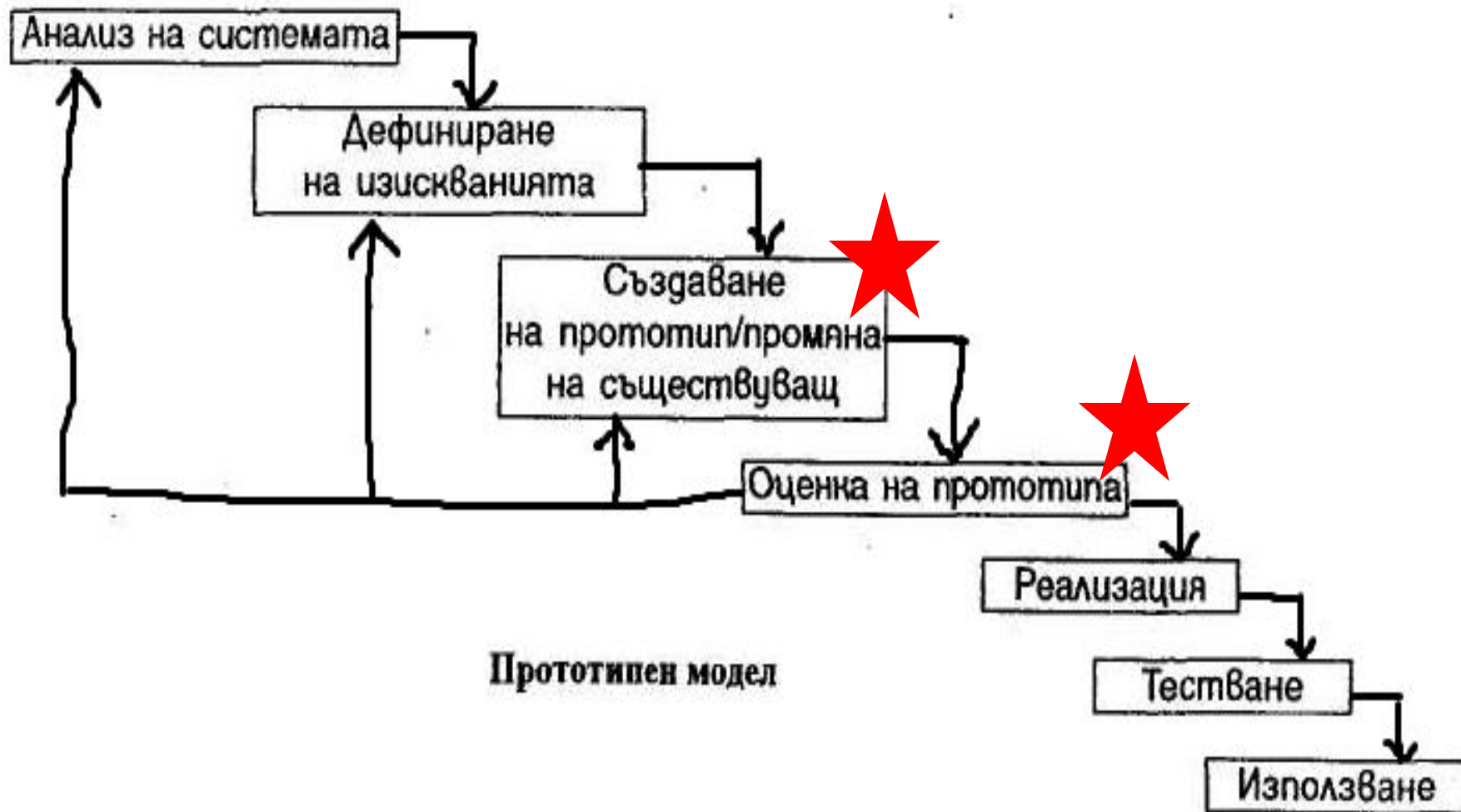
1.2. Разработване с прототипиране

- Прототипиране (дефиниция): съвкупност от дейности, осигуряващи ранно демонстриране на някои свойства на софтуера, който трябва да бъде създаден.
- Цел: Да се определят изискванията към разработваната софтуерна система.

Забележка:

- Прототипния модел на ЖЦ на софтуера и неговите фази бяха вече дефинирани в Тема 1.
- Фокусът тук е върху изпълнението на основните фази, свързани с **създаване, оценка и използване на прототипа**, с оглед вече на самото производство на софтуера.

Прототипен модел на ЖЦ - фази



Създаване на прототипа. Включва:

- **Проектиране на прототипа** – т.е избор на функциите и свойствата, които прототипът ще изяви:
 - Вертикален избор - само някои ф-ии и свойства, но те се ще се реализират в техния предполагаем окончателен вид;
 - Хоризонтален избор - всички функции и свойства, но ще се реализират не напълно.
- **Разработка на прототипа** – реализират се проектираните функции (цялостно някои от функциите или частично всички функции – това зависи от избора в предната стъпка);

- **Оценяване на прототипа** - важна стъпка, защото осигурява обратна връзка. Прави се **от експерти** или потребители на системата, въз основа на **документ**, описващ критериите за оценка;
- **Използване** - Прототипът или се отхвърля, или се използва като база за разработването на целевата система - последната трябва да притежава всички функции и свойства, демонстрирани в прототипа.

Забележка: Поради идеята на този модел на ЖЦ, а именно най-бърза разработка на прототипа, много често, прототипът не може да се вгради в целевата система поради следните недостатъци:

Разработване с прототипиране. Недостатъци:

- За да се разработи бързо прототипа, може да е избрана неподходяща за целевата система операционна или хардуерна среда;
- Някои съществени за системата характеристики могат да бъдат игнорирани в прототипа, но да са задължителни за крайния продукт;
- Поради многократното модифициране на прототипа, той може да е с ниска съпровождаемост.

Видове прототипиране:

- **Изследователско** - изяснява изискванията към целевата система, чрез представяне на няколко алтернативни решения;
- **Експериментално** - проверява дали избрано решение на даден проблем е подходящо;
- **Еволюционно** - разработването се осъществява в динамична среда и непрекъснато променящи се изисквания (най-отдалечено от оригиналното разбиране). Крайният продукт се създава като последователност от инкременти, всеки от които е прототип за следващия.

За да бъде създаден максимално бързо прототипа, трябва да се използват и подходящи методи и средства.

Методи за създаване на прототипа:

- **Езици от четвърто поколение (4GL)** - дават възможност на разработчика бързо да генерира изпълним код и затова са подходящи за създаване на прототипи (н.р генератори на програми, езици на заявките и генератори на отчети в БД и др.);
- **Използване на готови софтуерни компоненти;**
- **И др.**

1.3. Разработване с участието на потребителя

Това е подход за разработване, навлязал след началото на 90-те години на миналия век.

- **Включва:** съвкупност от методи, техники и целенасочени действия, осигуряващи активното участие на потребителя в проектирането и създаването на софтуерната система, която той ще използва.
- **Цел:** Да се повиши активността и съответно отговорността на потребителя за качеството на програмния продукт, като той се включи активно и в някои междинни фази на разработка.

Има 2 вида софтуерни проекти:

- **Автономни СП:** Проекти без конкретен възложител. Обикновено до идеята за създаването им се достига след маркетингово проучване.
- **Възложени СП:** Конкретен възложител, който финансира разработката и след завършването и става неин собственик.
 - Класическите подходи в този случай предвиждат участие на потребителя само в началната и крайната фази на разработване на софтуера, свързани с разработка на изискванията и с оценка и приемане на ПП.
 - Алтернативата: **разработване на ПП с участие на потребителя (лице, което ще използва софтуера)**. Една възможна практика на такова разработване е **еволюционно прототипиране**, тоест разработването на ПП се разглежда като създаване на последователност от междинни ПП (**от Разработчика**), всеки от които е разширение на предния; Всеки така разработен ПП се тества в реални условия (**от Потребителя**).

2. Технологии за „Разработване с мултиплициране”



2.1 Подход на Повторно Използване (ППИ или Re-use подход)

- **Същност:** ППИ е съвкупност от планирани и систематични дейности, насочени към **максимално използване на съществуващи софтуерни елементи** в процеса на създаване на нов софтуер.
- **Цел:** намаляване на сроковете и стойността на софтуерните разработки.

Критерии за класификация

- Софтуерните елементи, които се преизползват могат да се класифицират според своята:
 - **същност**
 - **обхват**
 - **начин на осъществяване**
 - **използвана техника**
 - **начин на използване**
 - **вид на използвания софтуерен продукт**
- и др.

Класификация по същността им

- **идеи или концепции:**

- алгоритми,
- методи,
- техники или
- формални модели.

- **софтуерни компоненти:** Това са най-често използваният тип елементи.

Могат да бъдат:

- **цели приложни системи, Н.р** реализация на една и съща програмна система на различни платформи (т. е. в различни съчетания на операционна и хардуерна среда).
- **подсистеми:** такива, които реализират съвкупности от функции, които са с универсално предназначение. Например, подсистема за обработка на грешките, за реализиране на вградена помощна функция, за прилагане на съвкупност от софтуерни метрики, за натрупване на статистическа информация и др.
- **програмни модули, функции или други обособени програмни части;**
- **процедури или умения,** свързани с процесите на създаване на софтуер. В тази група е know-how информация, която може да бъде под формата на наблюдения, препоръки, експертно знание и др.

Класификация по обхват

В зависимост от **формата и докъде** се простира повторното използване, то може да бъде вертикално или хоризонтално.

- **вертикално повторно използване** - в избрана приложна област се създават основни модели, които се прилагат във всички разработвани за тази област софтуерни системи.
- **хоризонталното повторно използване** - създават се универсални компоненти, които могат да се вграждат в програмни системи за различни приложни области, например средства за управление на бази от данни, за разпределени среди, за изграждане на графичен потребителски интерфейс и др.

Класификация по начина на осъществяване

- **планирано и систематично** повторно използване. В съответната софтуерна организация са регламентирани основните принципи и процедури за осъществяване на този подход.
- **инцидентно (ad-hoc)** повторно използване. За конкретен софтуерен проект се търсят съществуващи софтуерни елементи, удовлетворяващи формулирани изисквания.

Класификация по използвана техника:

- **сглобяващо ПИ**. Съществуващи софтуерни компоненти се вграждат като основни блокове в софтуерните системи.
- **генериращо ПИ**. Използват се идеи или концепции само на спецификационно ниво, след което чрез специално разработени генератори на програми автоматично се създават съответните програмни части.

Класификация по начина на използване:

- **без промяна**. Съществуващи софтуерни елементи се прилагат без изменения. Този вид повторно използване трябва да се предпочита, защото се основава на проверено качество и минимизира усилията за съпровождане.
- **с модифициране**. В този случай се изисква допълнително специфициране и реализация на измененията и тестване, за да се провери коректността на получения вариант.

Класификация по вида на използвания софтуерен продукт

- Могат да се пре-използват:
 - спецификации,
 - проекти,
 - фрагменти от програмен текст,
 - документация,
 - тестови примери и др.

За успешно прилагане на ППИ

- **Осъзнаване на необходимостта от прилагане на ПИ**
подхода: необходимостта от въвеждане на подхода трябва да бъде осъзната от ръководството на организацията, което да планира, финансира и управлява разработката на софтуер. Мотивите за ПИ могат да бъдат **подобряване на качеството на създавания софтуер**, доколкото се използват елементи с проверени вече свойства, и **по-бързо излизане на пазара с нови продукти**, тъй като се намалява времето за разработка.
- **Създаване и документиране на библиотека от компоненти за ПИ**
- **Обучаване на всички участници в разработката как да използват библиотеката**

Разработване чрез компоненти (Component-based development)

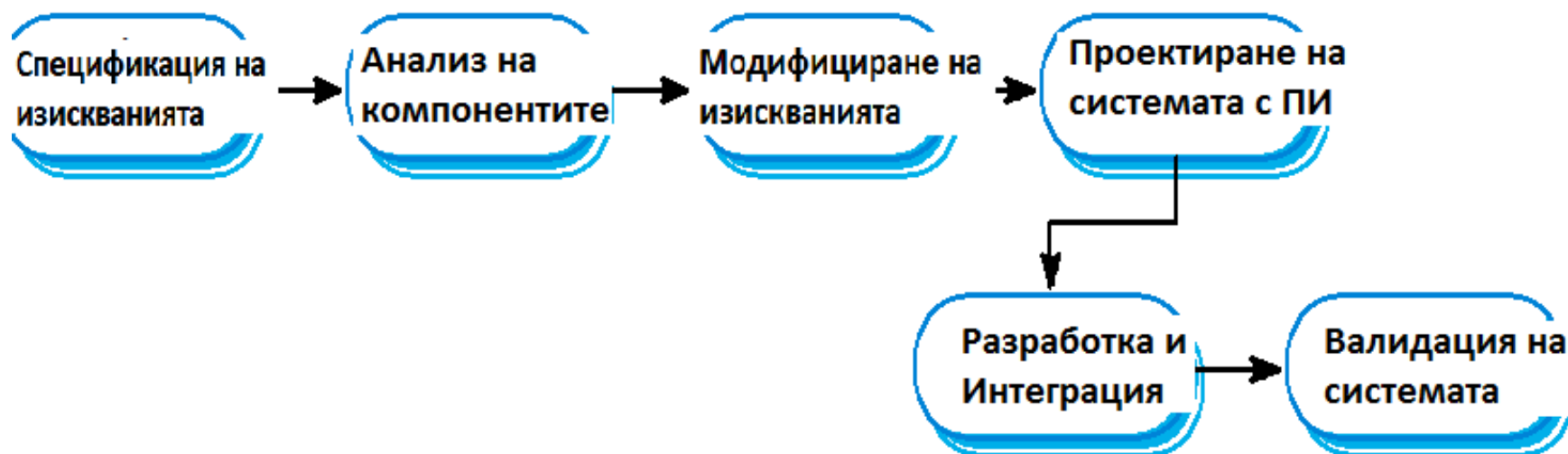
- **Същност:** Това е вариант на ППИ, при който системите се сглобяват от **съществуващи компоненти**.
- **Компонент:** софтуерна единица с функционалност и зависимости, изцяло определени от интерфейсите и. Може да се комбинира с други компоненти като изпълнима част, без да се взема предвид конкретната му реализация.

Характеристики на компонент

- Да е **стандартизиран** – да удовлетворява общоприети изисквания по отношение на интерфейси, документиране, вграждане, внедряване и др.
- Да е **независим**
- Да е **вградим**

За процеса на компонентно разработване

- **Специфичен е. Етапи на процеса:**
 - Анализ на компонентите;
 - Модификация на изискванията;
 - Проектиране на системата с многократно използване;
 - Разработка и интеграция



За процеса на компонентно разработване

- Чистото прилагане на процеса изисква преговори с потребителите, които трябва да са съгласни да използват софтуер, който е “слепен” от едно предлагано количество компоненти, които в някои случаи не отговарят съвсем на тяхните претенции, т.е да са склонни на компромиси.



За КОМПОНЕНТНИЯ ПОДХОД

- **Не може да бъде обявен за парадигма** (независимо от усилията за това) тъй като има много трудни за решаване проблеми, свързани с:
 - **Универсиализиране на процеса на компонентна разработка** (все още няма универсален модел за компонентно разработване на софтуер)
 - **Промисленото производство на компоненти,**
 - **Стандартизирано документиране,**
 - **Бързото им идентифициране и др.**

3. Разработване на надежден софтуер

- **Надеждността на функциониране** е динамична характеристика, която е свързана с **честотата на сригове в системата**.
- **Срив на системата** - ситуация по време на работа на софтуера, при която той започва да се държи по необичаен и неочакван начин.
- **Сривовете се дължат на дефекти в програмата.**

Какво е **дефект** в програмата?

- Всяко отклонение от изискванията към програмата ще наричаме **дефект**. Дефектите обикновено се дължат на една или няколко **грешки**.

**3 групи технологии за
„Разработване на
надежден софтуер“:**



3.1. Разработване на софтуер с минимизиране на дефектите в него

Свързва се с 2 подхода:

- ***fault-free software development***

и

- ***fault avoidance software development***
(н.р *Cleanroom SE*)

- *fault-free software* - софтуер без грешки.
- *fault avoidance software* - софтуер с минимално количество грешките

*Подход за разработване на **софтуер** **без грешки***

- Подходът предполага големи разходи.
- Оправдан е само за системи, функционирането на които е свързано с животаопасни (т.н **“критични”**) или изключително скъпо струващи дейности (управление на вредни производства и атомни електроцентрали, космически изследвания и др.).

Подход за разработване на *софтуер с* *избягване на грешките*

- ***fault avoidance software development***
— това е подход на разработване на софтуер, целта на който е да се намали възможността за внасяне на грешки в програмите.

Принципи за разработване на софтуер, с минимизиране на грешките в него

- а) създаване на прецизни, по възможност формални спецификации;
- б) използване на проектиране, което позволява капсулация (скриване) на информацията;
- в) използване на механизмите за осигуряване на качеството със систематична верификация и валидация на системата в определени контролни точки;
- г) планиране и осъществяване на тестването на системата така, че да се откриват и грешките, останали след верифицирането и валидирането;
- д) при кодиране - да се избягват, доколкото е възможно, конструкции на езиците за програмиране, които са потенциални източници на грешки.

Препоръки при кодирането, с цел - минимизиране на грешките в софтуера

- **да се избягва, доколкото е възможно:**
 - използването на числа с плаваща точка (има проблеми при сравняването им);
 - използване на указатели - директното обръщение към паметта крие опасности;
 - паралелелизъм (трудно се прогнозира ефектът от динамичното взаимодействие между паралелни процеси, трудно се тества, доколкото взаимодействията зависят от конкретните обстоятелства);
 - рекурсия (изисква висока програмистка квалификация); използване на системни прекъсвания (предава се управлението независимо от изпълнявания код).

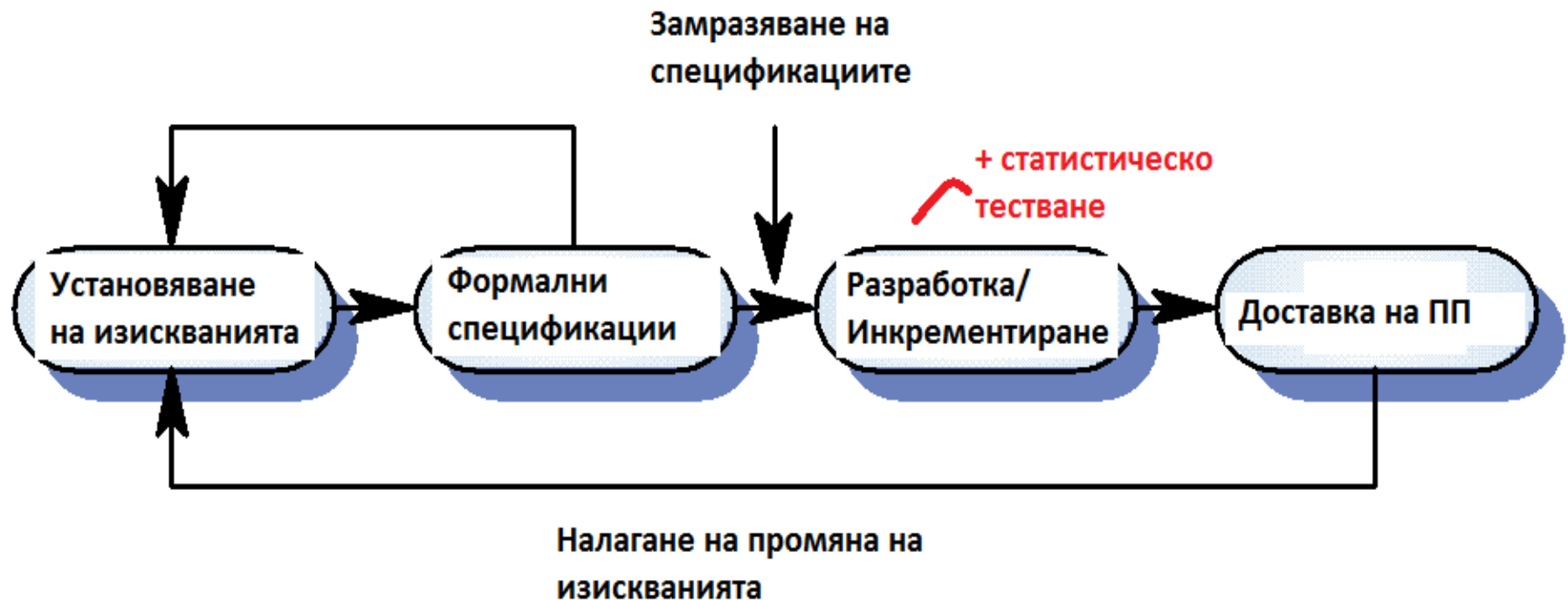
- да се използва принципът на “скриване на информацията”, който е заимстван от военните организации (need to know т.е. необходимост да знае - достъп до тази информация има само онзи, който се нуждае от нея, за да изпълнява задълженията си).
 - Прилагането на този принцип при разработването на софтуер означава, че всяка програмна компонента трябва да има достъп само до данните, необходими за реализираните от нея функции.
 - Тази препоръка се улеснява от въведената в съвременните езици за програмиране система от типове данни: класове, интерфейси и др.

Един пример за fault avoidance подход: «Cleanroom Software Engineering»

- **Cleanroom Software Engineering («чиста стая»)** — това е подход на разработка на ПО, предназначен за създаване на ПО със сертифицирано ниво на надеждност.
- **Cleanroom** е разработен от Харлан Милзом и няколко негови колеги от IBM (80-те години на 20-ти век). Основен принцип на cleanroom SE: **избягването на грешки (fault avoidance) е по-добро от тяхното отстраняване.**
- Названието «**Cleanroom**» е заимствано от електронната промишленост – така се наричат помещенията с висока степен на защита от замърсяване, така щото да се предотврати получаването на дефекти в процеса на производство на полупроводници.
- Подходът предполага използването на много **формални методи и методи за осигуряване на качество на софтуера.**

Cleanroom Software Engineering [3] се основава на:

- формални спецификации
- инкрементална реализация
- проектиране и имплементация на статистически тестове



3.2. Разработване на софтуер с приемливо ниво на грешки (fault tolerance software development)

- ***fault tolerance software*** (софтуер с приемливо ниво на грешки) - софтуер, който е проектиран и програмиран така, че продължава работата си и при наличие на грешки.

- Софтуерните системи, базирани на този подход на разработване, трябва да реализират следните **основни функции**:
 - а) **прогнозиране или откриване на грешки**, по възможност преди да са довели до срыв на системата;
 - б) **оценка на пораженията след срыв на системата** (т. е. намиране на засегнатите части);
 - в) **възстановяване след срыв**. Това може да се постигне чрез отстраняване на повредите или чрез връщане към някое предишно устойчиво състояние на системата.
 - г) **локализиране и отстраняване на грешките**, предизвикали срыва на системата.

Методи за разработване на надежден софтуер

а) Принцип на повтарящите се елементи

- Осигуряването на надеждно функциониране може да се осъществи чрез **принципа на повтарящите се елементи**.
- Той се състои в това, че за извършване на една и съща функция (или група функции), в ПС се реализират повече от 1 (обикновено нечетен брой) компоненти.
- Всяка компонента изпълнява определената функция (функции), след което се сравняват получените резултати. По-нататъшната работа на ПС продължава с най-често повтарящия се резултат.

б) N-версийно програмиране

- Осигуряването на надеждно функциониране може да се осъществи също и чрез ***N-версийно програмиране.***
- Избира се критична за функционирането на системата компонента. Тя се възлага за проектиране и програмиране на различни групи и всички версии се включват в системата. Когато е необходимо, те се извикват (паралелно или последователно) и изпълнението на ПС продължава с най-често срещания резултат.

в) Възстановяващи блокове

- Понякога в програмните системи се вграждат т. нар. *възстановяващи блокове* — програмни единици, проверяващи за срыв на системата и включващи алтернативен код, позволяващ повторение на процедурата, ако има съмнение за наличие на дефект.

Използването им може да се илюстрира със следния пример:

- За решаването на един и същ проблем са създадени три компонента, реализиращи три различни алгоритъма.
- Разработена е и тестваща компонента, която проверява правилността на получаваните резултати.
- За определени входни данни се изпълнява компонентата, реализираща алгоритъм 1, и резултатите от изпълнението се анализират от тестващата компонента. Ако те са правилни, изпълнението продължава. Ако има съмнения за грешки, същите входни данни се подават на компонентата, реализираща алгоритъм 2, и резултатите отново се насочват към тестващия модул. При неуспех аналогична процедура се повтаря с третата компонента. При неуспех и след нейното изпълнение се включва специална програмна част.

г) Обработка на изключенията (exception handlers).

- Изключение се нарича **НЕОЧАКВАНО СЪБИТИЕ**, което възниква по време на изпълнение на програмата и заради което програмата не може да бъде изпълнена успешно.
- В някои софтуерни системи се създават отделни **компоненти за обработване на изключенията (exception handlers)**. Така могат да се управляват реакциите на системата при настъпване на аварийни ситуации. Най-простата реакция е извеждане на съобщение и преустановяване на изпълнението. Но са възможни и по-сложни последователности от действия.
- В езиците за програмиране има вградени възможности за реализация на обработване на изключенията.

Защитно програмиране

- Този подход на разработване софтуер се основава на предположението, че в **програмите има неоткрити грешки**.
- Затова още при програмирането се вграждат механизми за откриване на грешката, оценка на засегнатите програмни части и отстраняване на последствията.

Механизми за откриване на грешки и възстановяване на последствията

- Една възможна техника е “**определяне на критични за състоянието на системата променливи**” и текущо проверяване на стойностите им чрез твърдения, описващи условия или пресмятане на контролни суми.
- Например в СУБД този принцип се реализира, като промените в базите от данни се съхраняват междинно в някакъв буфер и се внасят в съответната база само след успешно завършване на поредната транзакция.

Механизми за откриване на грешки и възстановяване на последствията

- Друг пример на защитно програмиране е “**използването на контролни точки**”.
- Този принцип се прилага най-често при сложни изчислителни задачи:
 - Изчисленията се извършват поетапно.
 - Всеки етап завършва с контролна точка, в която се запомнят междинните резултати.
 - Във всяка контролна точка изпълнението може да бъде прекъснато и да продължи след време от достигнатото в момента състояние.
 - При срыв на системата се осъществява връщане към последното запомнено състояние.

Въпроси

- Гъвкави технологии: видове и принципи на гъвкави технологии; екстремно програмиране: цел и същност.
- Гъвкави технологии: видове и принципи на гъвкави технологии; разработка с прототипиране: цел и същност. Видове прототипиране.
- Гъвкави технологии: видове и принципи на гъвкави технологии; разработка с участието на потребителите: цел и същност.
- Подход на Повторно Използване (ППИ или Re-use подход): същност, цел, класификации на елементите за пре-използване. Компонентен подход.
- Какво означава разработване на софтуер с приемливо ниво на грешки (fault tolerance software development) и какви са основните функции, които трябва да притежават системи, имплементиращи този подход?
- Кои са подходите за разработка на софтуер с минимизиране на дефектите и на какви принципи почива разработката на такъв софтуер? Какви препоръки би следвало да се съблюдават? Какво е Cleanroom?
- Какво е защитно програмиране? Обяснете механизмите за откриване на грешки и възстановяване на последствията при защитно програмиране?

Основна литература:

1. Нели Манева, Аврам Ескенази, Софтуерни технологии. Анубис София 2001
2. Нели Манева, Аврам Ескенази, Софтуерни технологии. КЛМН, София 2006
3. Sommerville: Software Engineering ,
9. ed. Addison-Wesley, 2011,
[http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20\(intro%20txt\)%20-%20I.%20Sommerville%20\(Pearson,%202011\)%20BBS.pdf](http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20(intro%20txt)%20-%20I.%20Sommerville%20(Pearson,%202011)%20BBS.pdf)